CSCI 5561 Homework 2

Quenton Ni
ni000100@umn.edu

1 Introduction

In this project, I implemented flow matching to generate samples from a two dimensional distribution and MNIST digits. I first examined the learned velocity field on eight Gaussian clusters, then verified image training by overfitting a small batch of digits. After, I compared network architectures, training strategies, and numerical samplers to improve unconditional image generation.

2 Two-Dimensional Flow Matching

I selected a target distribution consisting of eight Gaussian clusters arranged on a circle of radius 2. Each cluster had a standard deviation of 0.12. The velocity model was an MLP with dimensions $3 \to 128 \to 128 \to 2$, SiLU activations, and 17,282 parameters. The input contained the two spatial coordinates and time. I trained the model for 5,000 updates using Adam, a learning rate of 10^{-3}, and batches of 256 points. Sampling used 100 fixed Euler steps.

The interpolated input is $x_t = (1-t)x_0 + tx_1$, where x_1 is a training sample, x_0 is Gaussian noise and t is sampled uniformly from $[0, 1]$. $t = 0$ corresponds to noise and $t = 1$ to data. The training objective is

$$Loss = \text{MSE}\big(v_\theta(x_t, t),\, x_1 - x_0\big). \qquad (1)$$

As seen in Figure 1 the generated samples roughly align to the true samples after 100 euler steps. As seen in Figure 2 velocities are concentrated toward the middle of the image at small time steps before converging at the different centers at higher time steps. Figure 3 show the path of 128 different noise to target samples. It corroborates the velocity fields with points initially heading toward the center before converging to their respective target centers.

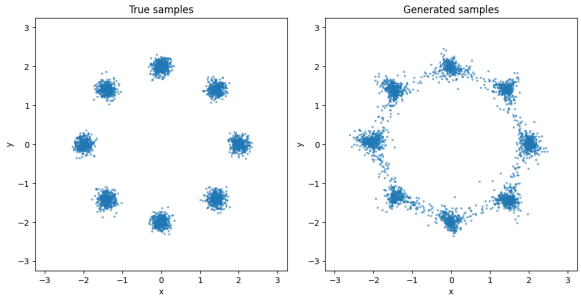


Figure 1: Scatter plot comparing generated and true samples

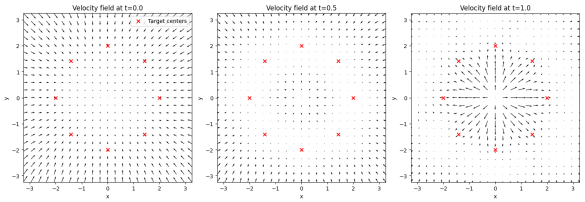


Figure 2: Velocity field visualization

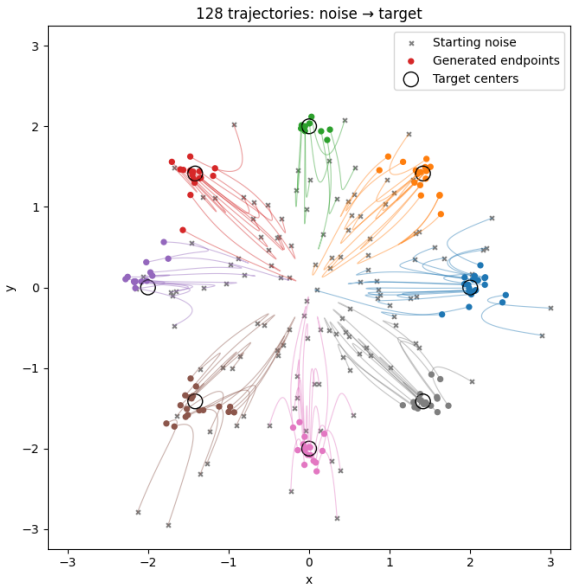


Figure 3: Trajectory plot

3 Image Model and Training Diagnostics

For image generation, I represented each MNIST image as a $1 \times 28 \times 28$ tensor and normalized pixel values to [-1,1]. The image network predicts a velocity tensor with the same shape as its input. The same loss as equation 1 is used.

This run trained on only 16 MNIST images. This run used a timestep conditioned U-Net with channel widths 16, 32, and 64, a 32-dimensional time embedding, and 124,145 parameters. I trained for 10,000 updates using Adam at 10^{-3}.

I used the same 16 starting noise images to generate samples after 100, 1,000, and 10,000 updates, using 50 Euler steps. Early outputs were noisy and ambiguous, while later outputs had recognizable digits with patterns resembling the small training set. The training loss decreased from 0.3114 at update 500 to 0.0937 at update 10,000. This experiment showed that the image model could fit a small distribution before moving to full data training.

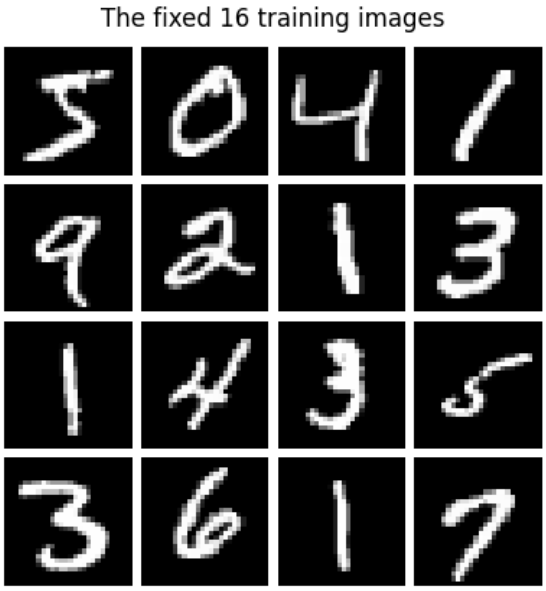


Figure 4: Fixed training images.

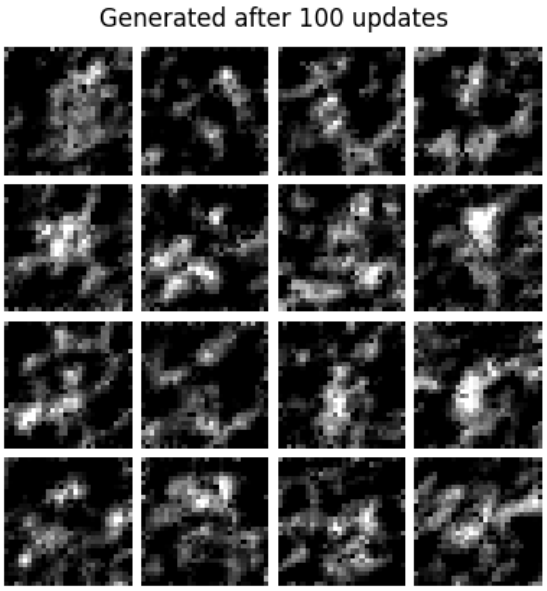


Figure 5: Early generated samples from overfit model

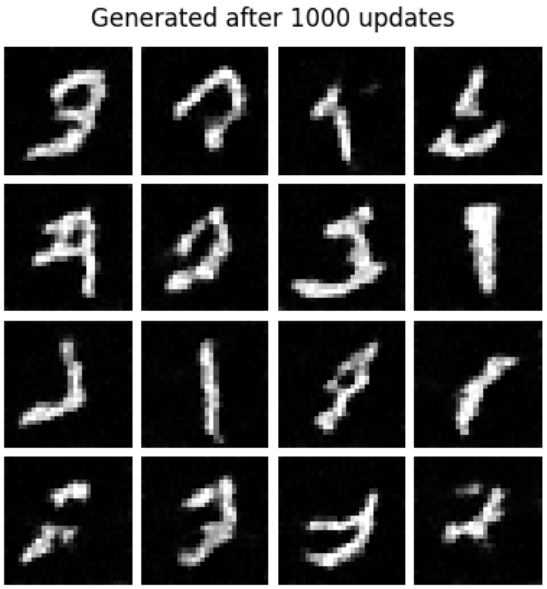


Figure 6: Generated samples from overfit model after 1000 steps

4 Unconditional Image Generation

4.1 Architecture and Training

The final model is a 5,358,785-parameter U-Net with channel widths 64, 128, and 256. It contains two residual blocks at each encoder, bottleneck, and decoder level, with encoder features concatenated into the corresponding decoder levels. Each block uses four group GroupNorm, SiLU activations, two 3×3 convolutions, and FiLM time con-

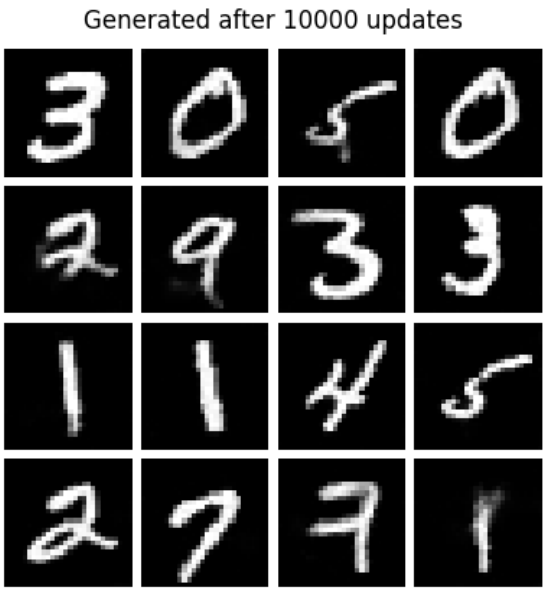


Figure 7: Generated samples from overfit model after 10000 steps

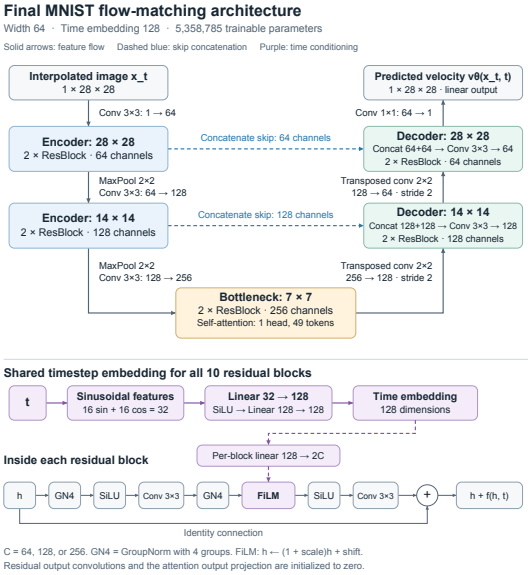


Figure 8: Illustration of final model architecture.

ditioning.

Time is encoded using 32 sinusoidal features and a learned $32 \to 128 \to 128$ embedding. Single-head self attention happens at the 7×7 bottleneck. The final 1×1 convolution predicts velocities without an output activation.

I trained on all 60,000 MNIST training images, normalized to $[-1, 1]$, using batch size 128 and Adam optimizer. The learning rate warmed up over 200 updates to 3×10^{-4}, then followed cosine decay to 10^{-5}. I trained for 200 epochs(93,800 steps). I also used EMA with a decay of 0.999. The run used a NVIDIA RTX PRO 6000 Blackwell Server Edition GPU and took 25.58 minutes to train. Epoch 180 was selected with the highest recognition score among checkpoints while satisfying full coverage.

4.2 Performance Experiments

Increasing width from 16 to 32 improved the ten-epoch baseline with a 50 Euler sampler from 47.85% to 55.30%, although neither passed coverage. Later experiments used EMA weights, fixed evaluation noise, and 25 Heun sampling. Table 1 summarizes the results.

Greater width and revised residual blocks gave the largest improvements in the 100 epoch comparisons. Increasing the time embedding gave only a small gain. Longer training improved performance. Constant learning rate and optimal transport pairing showed no clear advantage. OT reduced training MSE, but changing the pairing makes its loss directly incomparable with random pairing. For reference, these are all single run observations rather than averages across multiple runs.

Configuration	Epochs	Recognition
Scheduled width 32	100	84.85%
Scheduled width 64	100	86.90%
Revised residual blocks	100	88.75%
Time embedding 128	100	88.90%
Longer scheduled run	200	90.85%
Constant learning rate	100	89.10%
Minibatch OT	100	88.95%

Table 1: Comparing results using EMA and Heun 25. All results have full coverage.

On a separate fixed checkpoint, I tested different sampling strategies. Heun 50 achieved 90.55% in 5.47 seconds, compared with 89.75% in 5.52 seconds for Euler 100. Heun 100 reached 90.70% but took 10.99 seconds. I selected Heun 50 over Heun 100 due to the marginal difference in performance over the large difference in latency. The final step of the Heun sampling uses Euler, giving 99 network evaluations per sample.

Figures 9 and 10 show two earlier network tests. Both trained for 10 epochs using Adam at 10^{-3} and batch size 128. The first used ordinary convolutional blocks with EMA, while the second used residual blocks without EMA. Both produced somewhat recognizable digits with some samples containing ambiguous shapes. These configurations provided earlier comparison points for the final model, which combined residual processing and EMA with GroupNorm, FiLM, and attention.



Figure 9: Earlier width 32 U-Net with EMA decay 0.995 after 10 epochs, sampled using 50 Euler steps



Figure 10: Earlier width-32 residual U-Net after 10 epochs, sampled using 50 Euler steps.

Epoch	MSE	Recognition	Minimum count
1	0.2055	10.90%	0
20	0.1675	85.35%	138
180	0.1571	90.90%	145

Table 2: Final run progression.

4.3 Training Progress and Final Evaluation

I used identical starting noise throughout training to evaluate recognition and coverage. Table 2 shows different checkpoints from the final run.

The loss curve drops sharply near the beginning and levels out, giving minor improvement to loss over time. Recognition improved over time, even after the loss leveled out. There was a steady improvement until around epoch 80 where recognition score fluctuated in the high 80% low 90% range.

Final evaluation used 2,000 Gaussian starting samples, EMA, and Heun 50. Generation used no class labels and every sample was retained. Recognition is the fraction of images whose maximum classifier probabilities exceeds 0.99. Coverage counts all predictions and requires at least 140 examples of every digit.

The selected checkpoint achieved 90.90% recognition with full coverage. Predicted digit counts from 0 through 9 were 203, 259, 203, 215, 212, 145, 196, 170, 216, 181. I saved the complete batch as `samples.npy`, a uint8 array of shape $(2000, 28, 28)$.

Of the 2,000 generated samples, 182 (9.10%) had a max classifier probability below 0.99. Figure 11 shows 16 low confidence examples. The examples include irregular strokes that make the digits less clear. However, some remain visually recognizable. For example, the first image in the second row looks like a 1, but its classifier confidence is only 0.8982. This shows that failing the confidence threshold does not necessarily mean a sample is visually unrecognizable.

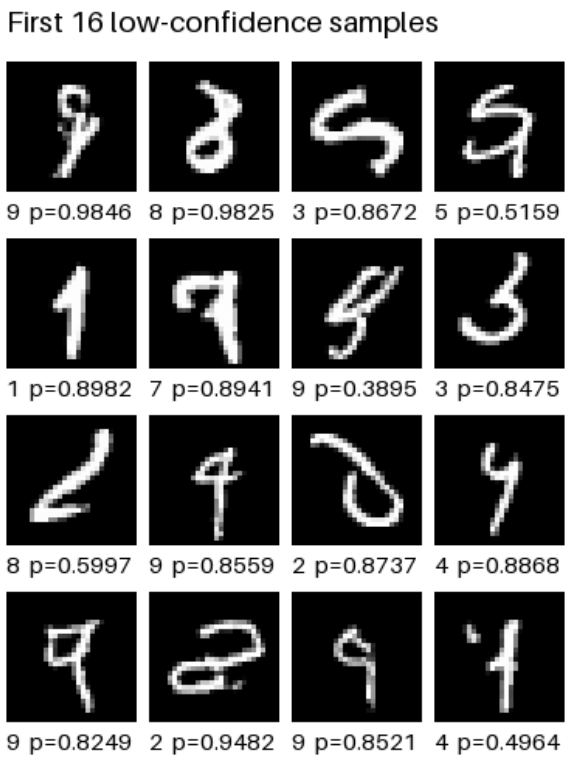


Figure 11: Low confidence examples from the evaluation. Labels show the predicted digit and its probability.

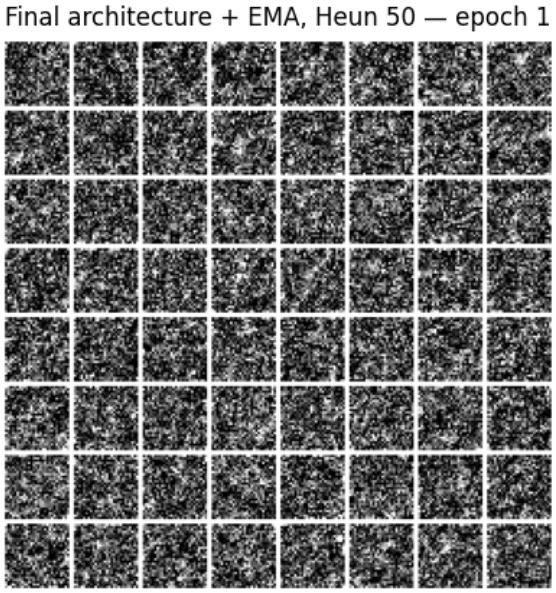


Figure 12: Sample of output from final architecture at epoch 1.

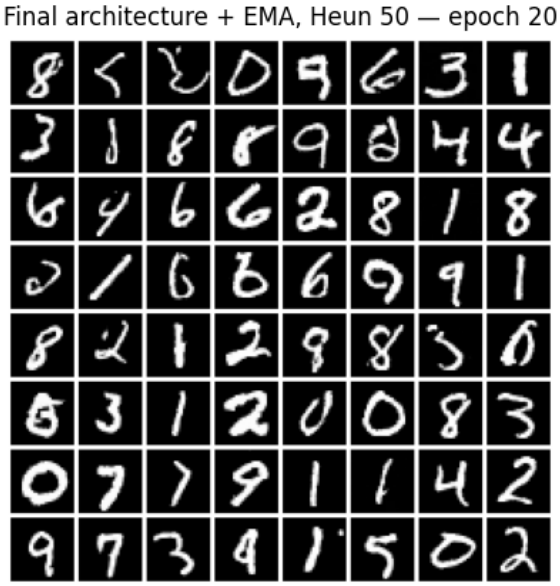


Figure 13: Sample of output from final architecture at epoch 20.

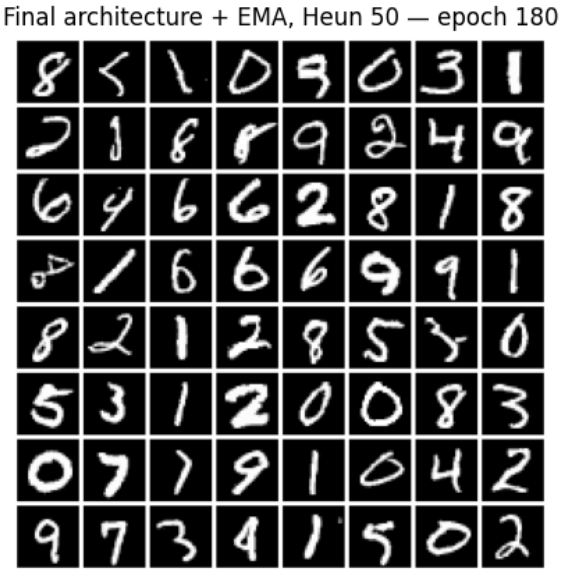


Figure 14: Sample of output from final architecture at epoch 180.

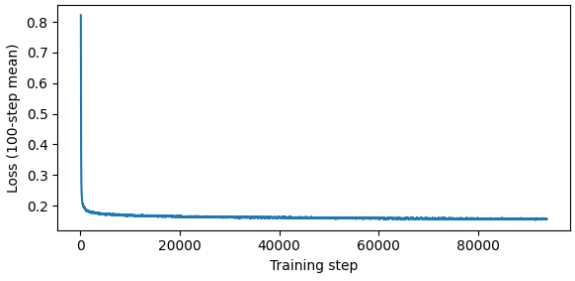


Figure 15: Loss graph from training final model.